

Conceitos de Containers e Docker

Fundamentos de Containerização

AWS Certified Developer Associate

Instrutor: Marlon Anesi

O que são Containers?

Definição

Unidade padrão de software que empacota código e todas as suas dependências

Características Principais

- Isolamento: Aplicação isolada do sistema host
- Portabilidade: Executa consistentemente em qualquer ambiente
- Leveza: Compartilha o kernel do OS, usa menos recursos que VMs
- Startup rápido: Inicia em segundos (vs minutos para VMs)

Benefícios para Desenvolvedores

- Consistência: 'Funciona na minha máquina' resolvido
- Eficiência: Densidade maior - mais containers por host
- Versionamento: Imagens versionadas e imutáveis

Docker: Conceitos Fundamentais

O que é Docker?

Plataforma líder para desenvolvimento, distribuição e execução de containers

Componentes Principais

- Docker Image: Template read-only com instruções para criar container
- Docker Container: Instância executável de uma imagem
- Dockerfile: Arquivo de texto com instruções para build da imagem
- Docker Registry: Repositório de imagens (Docker Hub, ECR)

Comandos Essenciais

- `docker build`: Cria imagem a partir do Dockerfile
- `docker run`: Executa container a partir de uma imagem
- `docker push`: Envia imagem para registry
- `docker pull`: Baixa imagem do registry

Containers vs Máquinas Virtuais

Aspecto	Containers	Máquinas Virtuais
Startup	Segundos	Minutos
Tamanho	MB	GB
Isolamento	Nível de processo	Sistema operacional completo
Overhead	Baixo (compartilha kernel)	Alto (SO completo)
Densidade	Centenas por host	Dezenas por host
Portabilidade	Muito alta	Limitada ao hypervisor

Quando usar cada um?

- Containers: Microsserviços, aplicações cloud-native, CI/CD, ambientes de desenvolvimento
- VMs: Isolamento completo de SO, cargas legadas, múltiplos SOs no mesmo host

Casos de Uso de Containers

Desenvolvimento Local

- Ambiente consistente
- Mesmas versões de dependências
- Setup rápido para novos devs
- Evita conflitos de versões
- Facilita onboarding de time

Arquitetura de Microsserviços

- Deploy independente de serviços
- Escalabilidade granular
- Isolamento de falhas
- Tecnologias heterogêneas
- Facilita CI/CD

Boas Práticas com Containers

- Mantenha imagens pequenas: Use imagens base leves (alpine)
- Um processo por container: Não trate container como VM
- Use .dockerignore: Evite arquivos desnecessários na imagem
- Multi-stage builds: Reduza tamanho final da imagem
- Tags específicas: Nunca use 'latest' em produção

Pontos-chave para o Exame

1. Conceitos Fundamentais

- Container = aplicação + dependências isoladas
- Compartilha kernel do OS (diferente de VMs)
- Startup rápido e uso eficiente de recursos

2. Docker: Componentes

- Image: Template imutável
- Container: Instância executável da imagem
- Dockerfile: Instruções para build
- Registry: Repositório de imagens (ECR na AWS)

3. Vantagens vs VMs

- Mais leve e rápido (segundos vs minutos)
- Maior densidade (centenas vs dezenas por host)
- Portabilidade: executa em qualquer ambiente

LEMBRE-SE: Se a questão menciona 'executar aplicações de forma isolada e portátil' > pense Containers!
